Pontificia Universidad JAVERIANA Colombia

Evaluator-optimizer

Semana 11 · Iterar una respuesta contra un evaluador
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

- Fundamentos con LangChain — Semanas 1–3 ✓
- RAG y evaluación — Semanas 4–6 ✓
- Model Context Protocol — Semana 7 ✓
- **LangGraph y patrones agénticos** — Semanas 8–11
- LLMOps y despliegue — Semana 12

Semana 11 · Evaluator-optimizer

- De detectar problemas a corregirlos
- Un ciclo con tope de intentos
- Un evaluador con rúbrica explícita
- Retroalimentación al generador
- Leer el historial de intentos
- Cuándo vale la pena el ciclo

Las ideas que traemos

Repaso

Semanas 6, 8 y 10

1. Semana 6: un juez con rúbrica mide una respuesta; se calibra antes de confiar.
2. Semana 8: tres revisores en paralelo detectaban problemas en un borrador.
3. Semana 10: el plan del orquestador puede fallar y hay que leerlo.

Una respuesta que debe cumplir reglas

El problema de la semana

Una rúbrica explícita

«Perdí mi empleo y no puedo pagar la cuota. ¿Qué opciones tengo?»

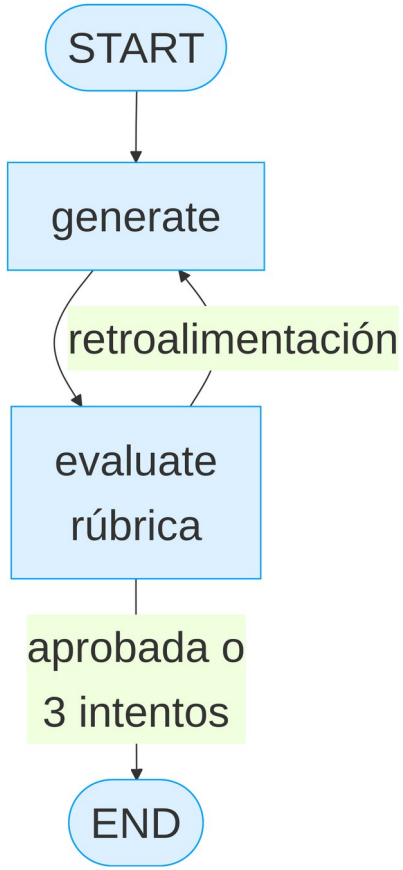
La respuesta debe usar las cifras de la política, citar cada dato, no prometer aprobaciones y tener **como máximo 90 palabras**.

Generar, evaluar, corregir

L22 · secciones 1 a 3

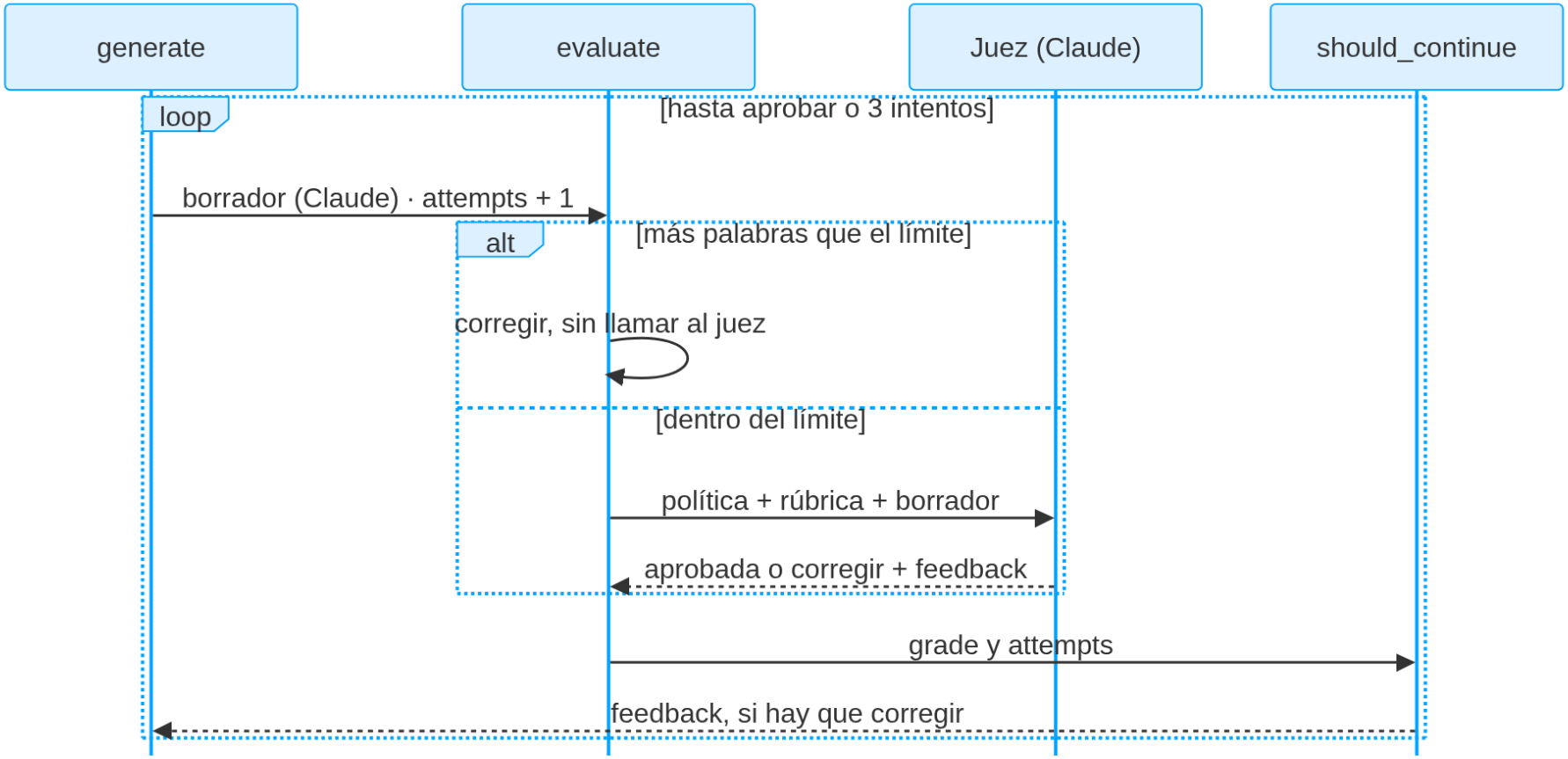
Un ciclo que siempre termina

La arista condicional vuelve a `generate` hasta aprobar o llegar a 3 intentos.



El ciclo en el tiempo

Diagrama de secuencia



Hasta aprobar o 3 intentos. Lo que el código puede contar se revisa antes del juez; el juez solo lee lo que necesita criterio.

La rúbrica y el evaluador

L22 · sección 2

Salida estructurada

El evaluador ve la política y la rúbrica; el generador, su retroalimentación.

```
RUBRIC = """\
Aprueba solo si la respuesta cumple todo:
1. Las cifras y plazos coinciden con la política.
2. Cada dato lleva su cita [archivo, p. N].
3. No promete aprobaciones ni da asesoría personalizada.
4. Tiene como máximo 90 palabras.
"""

class Feedback(BaseModel):
    """Evaluación de una respuesta a un cliente de ACME."""
    grade: Literal["aprobada", "corregir"]
    feedback: str = Field(
        description="Qué corregir, punto por punto.")
```

El ciclo con tope

L22 · sección 1 · sin modelo

Terminar siempre

Sin tope, un evaluador exigente puede gastar tokens sin fin.

```
MAX_ATTEMPTS = 3

def should_continue(state: DraftState) -> Literal["generate", "__end__"]:
    if state["grade"] == "aprobada" or state["attempts"] >= MAX_ATTEMPTS:
        return END
    return "generate"

builder.add_edge(START, "generate")
builder.add_edge("generate", "evaluate")
builder.add_conditional_edges("evaluate", should_continue)
```

El historial de intentos

L22 · ejecución guardada · 4 llamadas

Intento	Veredicto	Retroalimentación
1	corregir	Supera 90 palabras y queda cortada; falta una cita; revisar promesas
2	aprobada	—

¿Aprobada de verdad?

Leer la respuesta final

Evaluar al evaluador

1. La respuesta aprobada en el intento 2 tiene tres bloques con citas correctas.
2. Pero supera con claridad las 90 palabras de la rúbrica.
3. Un evaluador LLM también se equivoca: contar palabras es mejor hacerlo en código.
4. Combinar comprobaciones de código y juez, como en la semana 6.

Los modelos no cuentan palabras

Un hallazgo del skill agent-patterns

Contar en código

1. Con la rúbrica y el límite en el prompt, Haiku escribió 112, 96 y 96 palabras (sin citas) para un límite de 90.
2. La retroalimentación le decía la cifra exacta; aun así quedó por encima.
3. El tope de intentos terminó el ciclo: sin él, habría seguido gastando.
4. Lección: el conteo va en código, el límite deja margen, y se reporta cuando se llegó al tope.

Cuándo vale la pena

L22 · sección 4

Costo frente a calidad

1. Hay criterios explícitos que un evaluador puede comprobar.
2. La retroalimentación mejora el siguiente intento de forma medible.
3. Cada intento cuesta 2 llamadas: entre 2 y 6 en este ejemplo.
4. Si las reglas se pueden comprobar en código, mejor no gastar un evaluador LLM.

El mismo juez, dos usos

Evaluator-optimizer frente a la evaluación de la semana 6

	Evaluator-optimizer	Evaluación (semana 6)
Cuándo corre	En cada respuesta, antes de enviarla	Antes de publicar, sobre un dataset
Para qué	Corregir esta respuesta	Medir y comparar configuraciones
Referencia	La rúbrica y la política	Respuestas escritas por personas
Costo	2 llamadas por intento, en cada uso	Una vez por experimento
Riesgo	Aprobar lo que la rúbrica prohíbe	Un juez sin calibrar

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L22-EvaluatorOptimizer

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Construir el ciclo con un generador falso y comprobar que termina.
2. Escribir el evaluador con la rúbrica y salida estructurada.
3. Ejecutar el ciclo con Claude y leer el historial de intentos.
4. Añadir una comprobación de longitud en código y comparar.

El bloque de patrones agénticos

Semanas 8 a 11

Patrón	Notebook	Qué fija el código
Prompt chaining	L17	Los pasos y la compuerta
Routing	L18	Los especialistas
Parallelization	L19	Los revisores y el agregador
Human-in-the-loop	L20	Dónde se pausa y los límites
Orchestrator-workers	L21	El trabajador y la síntesis
Evaluator-optimizer	L22	El tope de intentos y la rúbrica

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. Todo ciclo necesita un tope.
2. La rúbrica debe ser explícita y comprobable.
3. El evaluador también se evalúa.
4. Elegir siempre el patrón más simple que resuelva el problema.

Pontificia Universidad JAVERIANA Colombia

Próxima semana

LLMOps y despliegue: el asistente de ACME como API con memoria persistente.
`src/agents/acme.py` · `src/api/main.py`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso